

01 Herencia

JERARQUÍA DE CLASES

Empleado (padre)

```
protected String nombre
protected double salario
constructor(nombre, salario)
toString()
```

↓ extends

Operario

```
private String turno
super(nombre, salario)
toString()
```

Directivo

```
private String dept
super(nombre, salario)
toString()
```

SINTAXIS CLAVE

```
public class Operario extends Empleado {
    private String turno; public Operario(String
nombre, double salario, String turno){
    super(nombre, salario); // 1º this.turno =
turno; } }
```

- 1 `super()` siempre en la **primera línea** del constructor hijo
Si no, error de compilación
- 2 `protected` en la clase padre → accesible en hijos sin getter
`private` no es heredable

02 Polimorfismo

LISTA POLIMÓRFICA

```
// La lista acepta cualquier Empleado
List<Empleado> lista = new ArrayList<>();
Empleado op = new Operario( "Gabriel", 3000,
"Tarde"); Empleado dir = new Directivo(
"Diego", 4000, "IT"); lista.add(op);
lista.add(dir);
```

RECORRIDO CON FOR-EACH

```
for (Empleado e : lista) {
    System.out.println(e); // → toString() }
```

CÓMO FUNCIONA

- La variable es de tipo `Empleado`
Pero el objeto real es `Operario` o `Directivo`
- Java llama al `toString()` del tipo real
No el del padre → dynamic dispatch
- `@Override` en cada hijo garantiza su propio `toString()`

SALIDA ESPERADA

```
> Operario{turno='Tarde', nombre='Gabriel',
salario=3000.0}
> Operario{turno='Tarde', nombre='María',
salario=3000.0}
> Directivo{departamento='IT', nombre='Diego',
salario=4000.0}
> Directivo{departamento='Logística',
nombre='Manuela', salario=3800.0}
```

03 toString() y errores

ESTRUCTURA CORRECTA

```
@Override public String toString() { return
"Operario{" + "turno=" + turno + '\n' + ",
nombre=" + nombre + '\n' + ", salario=" +
salario + // sin ' '}; }
```

ERRORES DEL EJERCICIO

- ✗ `"departamento'="` — apóstrofo antes del =
correcto → `"departamento="`
- ✗ `"salario="` abre comilla sin cerrar
salario es double → sin comillas simples

REGLA COMILLAS EN TOSTRING

TIPO	PATRÓN
String	"campo=" + x + '\n'
int / double	"campo=" + x
boolean	"campo=" + x
cierre	+ '}'

- ! `'\n'` es el carácter `'` escapado
equivale a escribir una comilla simple literal
- i `println(e)` llama a `e.toString()` automáticamente